Transfer Learning

Definition:
The ability of a system to recognize and apply knowledge and skills learned in previous tasks to novel tasks or new domains, which share some commonality

Purpose:
- To leverage knowledge from previously trained models for training newer models
- To accelerate the learning process
- To improve generalization
- To tackle dataset bottlenecks
  (Reduce the need for data related to the target task)

Transfer Learning for Deep Learning
Instead of training a DNN from scratch for the target task, we deploy the network that has already been trained on a different domain for a different Source task, we adapt it to our domain and target task

Variations:
- Same domain - Different task
- Different domain - Same task

If 2 domains are different, they may have different Feature Spaces
If 2 tasks are different, they may have different Label spaces (different conditional distributions)

Deep Transfer Learning

Strategies:
- Off-the-shelf pre-trained models as feature extractors
- Fine tuning Off-the-shelf pre-trained models

Pre-trained models:
They are the details of networks shared by teams / individuals to be made available to all. These models are usually shared in the form of the millions of parameters / weights the model achieved while being trained to a stable state

Off-the-shelf pre-trained models as feature extractors:
- The key idea is to simply leverage the weighted layers of the pre-trained model to **extract features**, but NOT to update the weights of the model layers during training with new data for the new task
- Approach: Use the output of 1 or more layers of a network trained for a different task as **generic feature extractors**. Then, train a new **shallow classifier** on these features

Note: Off-the-shelf pre-trained models are normally used when the source and target datasets are from the **same domain** and the target dataset is **small**

Fine tuning Off-the-shelf Pre-trained models:

- This is a more complex technique, in which we not only **replace** the last layer, but we also selectively **retrain** some of the previous layers
- We use the knowledge in terms of the global architecture of the network and use its states as a starting point for our retraining step
- Approach: Freeze (fix weights) some layers during retraining, or **fine-tune** others according to our needs

Note: Depends on the target task, we can:

- Freeze when the target data is scarce (avoid overfitting)
- Fine-tune when there are more target data
- Freeze means weights are not updated during the training (back-propagation)

There are 4 types of Deep Transfer Learning

Type 1: Domain Adaptation

Challenge:
The distribution of data in the target domain is different than in the source domain (Domain shift)

Questions:
How to overcome the differences between the domains so that a classifier trained on the source domain generalizes well to the target domain

1. Divergence based Domain Adaptation

- This works on the principle of **minimizing some divergence** based criterion between source and target distribution, hence leading to **domain-invariant features**
- Domain-invariant features refers to features that do not change significantly whether they are observed in the source domain or the target domain
- The goal is to learn such features that are effective and relevant for the task in both the training (source) and the application (target) environments
- By identifying and using domain-invariant features, a machine learning model can perform well on data from the target domain even though it was trained on data from the source domain
- Divergences are typically non-parametric and consist of handcrafted mathematical formulas that are not tailored to any specific dataset or task, such as classification, object detection or segmentation

2. Adversarial based Domain Adaptation

- This method is a promising approach to training robust DNNs and can generate complex samples across diverse domains
- It is a technique used in ML to **fool or misguide** a model with malicious input
- Approach: The model learn a discriminative mapping of target images to the source feature space (target encoder) by fooling a domain discriminator that tries to distinguish the encoded target images from source examples
- **Training Phase:**
    - The feature extractor and the discriminator are both active
    - The feature extractor learns to generate features
      (same feature extractor for both source and target data)
    - The discriminator learns to distinguish between the features coming from the source and the target domains
    - The adversarial training process aims to reach a point where the discriminator can no longer easily tell the difference between features from the 2 domains, indicating that the feature extractor is generating domain-invariant features
- **Prediction Phase:**
    - Only the feature extractor is used to process new data
    - The learned domain-invariant features are fed into the rest of the model to make predictions
    - The discriminator is discarded because its job of differentiating between domains is not relevant during the prediction stage

3. Reconstruction based Domain Adaptation

- This works on the idea of Image-to-Image translation
- The simplest model for Image-to-Image translation could be an encoder-decoder based network and use a discriminator to enforce encoder-decoder network to produce images that are similar to the source domain
- Then, train a source domain classifier based on source data
- During inference, the image of the target domain will be transmitted to the source domain classifier trained for prediction
    - **Encoder:** Compresses target image into a feature vector that captures essential details, ignoring domain-specific details
    - **Decoder:** Reconstructs the image to match the source domain's style, ensuring it fits the source domain
    - **Source Classifier:** Classifies the style-matched image using a model trained on source domain data

Note: Simply reconstructing both domains does not guarantee successful domain adaptation. The training design must encourage the learned features to transfer between domains

Type 2: Multi-task Learning

By giving a set of learning tasks (t1, t2, … ,tn), the learner co-learn all tasks simultaneously.
In other words, the learner optimizes the learning / performance across all of the n tasks **through some shared knowledge**

In this case, the learner receives information on multiple tasks at once without distinguishing between the source and targets

The generic multitask learning consists of 2 components:

- The shared layers
- The specific layers

MTL makes sense when

- Training on a set of tasks that could benefit from sharing lower-level features (Amount of data we have for each task is quite similar)
- Training a large network of neurons to do all tasks well (The size of the neural network must be taken into consideration if one wants to train it to adapt to many tasks)

Type 3: Zero-shot Learning

- This enables identification of classes that are not seen before by means of **transferring knowledge** from seen classes to unseen classes
- This knowledge transfer is usually done via utilizing **prior information from various auxiliary sources**, such as descriptions / semantic attributes / word embedding
    - The aim is learn a projection function from visual space (image features) to semantic space (word vectors / semantic embedding) using data from seen classes
    - Then, the unseen class image features are passed as input to the trained network and we get the corresponding semantic embedding as the output
    - Nearest neighbor search is performed in the semantic embedding space to find the closest match to the output of the network. Finally the label corresponding to the closest semantic embedding is predicted as the output label of the input image feature

Type 4: One-shot Learning

- This is a variant of Transfer learning where we try to infer the required output based on just 1 or a few training examples
- It emphasizes on **knowledge transfer**, which makes use of prior knowledge of learnt categories and allows for learning on minimal training examples

Problem of Naive CNN:

- A small training set is not enough to train a robust neural network
- Retraining the CNN every time the number of classes or dataset is increased is way too time-consuming and resource-intensive

Idea behind One-shot learning:

- Train a model to differentiate between the same and different pairs, then generalize these ideas to evaluate new categories
- Learning a similarity function (The DL model takes 2 images and returns a value that shows the similarity between the 2 images
- If the images contain the same object, the the neural network returns a value below a specific threshold, and if they are not the same object, it will be above the threshold

Method	Main idea	Example
Domain Adaptation — Divergence-based	Reduce the statistical difference between source and target domains.	Adapt a model trained on studio photos to outdoor photos.
Domain Adaptation — Adversarial-based	Train a model to fool a domain discriminator so the domains become harder to distinguish.	Make studio and outdoor image features look similar.
Domain Adaptation — Reconstruction-based	Reconstruct inputs or representations to learn useful features across domains.	Use an encoder-decoder to learn meaningful features from clear and blurry images.
Multi-Task Learning	Train one model to perform multiple tasks simultaneously.	Classify animals and predict their ages.
Zero-Shot Learning	Recognize a new class without labeled training examples of that class.	Recognize a zebra using its description, despite having no labeled zebra images during training.
One-Shot Learning	Learn to recognize a new class from just one labeled example.	Show the model one photo of a person, then ask it to recognize that person in other photos.